

Scikit-learn Models & fit() Cheat Sheet

A practical reference for choosing common scikit-learn models, understanding their **constructor parameters**, and knowing what arguments **model.fit()** accepts.

Important distinction: parameters passed when creating a model, such as `RandomForestRegressor(n_estimators=100)`, are different from the data passed to `fit(X, y)`. Most scikit-learn estimators use `fit(X, y, ...)`; optional arguments vary by estimator.

1. The basic scikit-learn workflow

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

fit() normally takes at least two positional inputs: X (features) and y (target). Some estimators also accept optional arguments such as `sample_weight`.

2. Core supervised-learning models

Model	Import	Main use	Typical fit signature
LinearRegression	from sklearn.linear_model import LinearRegression	Continuous target; roughly linear relationship.	fit(X, y, sample_weight=None)
Ridge	from sklearn.linear_model import Ridge	Linear regression with L2 regularization; useful with correlated features.	fit(X, y, sample_weight=None)
Lasso	from sklearn.linear_model import Lasso	Linear regression with L1 regularization; can drive coefficients to zero.	fit(X, y, sample_weight=None)
ElasticNet	from sklearn.linear_model import ElasticNet	Combination of L1 and L2 regularization.	fit(X, y, sample_weight=None)
LogisticRegression	from sklearn.linear_model import LogisticRegression	Classification; predicts class probabilities/classes.	fit(X, y, sample_weight=None)
DecisionTreeRegressor	from sklearn.tree import DecisionTreeRegressor	Nonlinear regression using decision rules.	fit(X, y, sample_weight=None)
DecisionTreeClassifier	from sklearn.tree import DecisionTreeClassifier	Classification using decision rules.	fit(X, y, sample_weight=None, check_input=True)
RandomForestRegressor	from sklearn.ensemble import RandomForestRegressor	Strong general-purpose tabular regression.	fit(X, y, sample_weight=None)
RandomForestClassifier	from sklearn.ensemble import RandomForestClassifier	Strong general-purpose tabular classification.	fit(X, y, sample_weight=None)
GradientBoostingRegressor	from sklearn.ensemble import GradientBoostingRegressor	Accurate nonlinear tabular regression.	fit(X, y, sample_weight=None)
GradientBoostingClassifier	from sklearn.ensemble import GradientBoostingClassifier	Accurate nonlinear tabular classification.	fit(X, y, sample_weight=None)
HistGradientBoostingRegressor	from sklearn.ensemble import HistGradientBoostingRegressor	Faster gradient boosting for larger tabular datasets.	fit(X, y, sample_weight=None)
HistGradientBoostingClassifier	from sklearn.ensemble import HistGradientBoostingClassifier	Fast gradient boosting classification.	fit(X, y, sample_weight=None)
SVR	from sklearn.svm import SVR	Regression with support vectors; good for smaller/medium datasets.	fit(X, y, sample_weight=None)
SVC	from sklearn.svm import SVC	Classification with support vectors.	fit(X, y, sample_weight=None)
KNeighborsRegressor	from sklearn.neighbors import KNeighborsRegressor	Regression based on nearby training samples.	fit(X, y)
KNeighborsClassifier	from sklearn.neighbors import KNeighborsClassifier	Classification based on nearby training samples.	fit(X, y)
MLPRegressor	from sklearn.neural_network import MLPRegressor	Neural-network regression for nonlinear patterns.	fit(X, y)
MLPClassifier	from sklearn.neural_network import MLPClassifier	Neural-network classification.	fit(X, y)

3. Constructor parameters you should actually learn

The following are the important model-creation parameters. You do **not** need to memorize every parameter. Learn the ones that control model complexity, regularization, or training behavior.

Model	Important constructor parameters	What they control
LinearRegression	fit_intercept, positive, n_jobs	Intercept; force non-negative coefficients; parallelism where applicable.
Ridge	alpha, fit_intercept, solver, max_iter	Regularization strength; intercept; optimization method; iterations.
Lasso	alpha, fit_intercept, max_iter, tol	L1 regularization strength; intercept; convergence.
ElasticNet	alpha, l1_ratio, max_iter, tol	Overall regularization and L1/L2 mix.
LogisticRegression	C, penalty, solver, max_iter, class_weight	Inverse regularization strength, regularization type, optimizer, iterations, class weights.
DecisionTree*	criterion, max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes	How splits are scored and how complex/deep the tree can become.
RandomForest*	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, bootstrap	Number of trees, tree complexity, feature sampling, and bootstrap sampling.
GradientBoosting*	n_estimators, learning_rate, max_depth/max_leaf_nodes, subsample	Number and strength of boosting stages, tree complexity, sampling, objective.
HistGradientBoosting*	learning_rate, max_iter, max_leaf_nodes, max_depth, l2_regularization	Learning rate, number of iterations, tree complexity, regularization.
SVR/SVC	C, kernel, gamma, degree, epsilon (SVR)	Margin/regularization, kernel shape, polynomial degree, regression tolerance.
KNeighbors*	n_neighbors, weights, algorithm, leaf_size, p, metric	Number of neighbors, distance weighting/metric, search method.
MLP*	hidden_layer_sizes, activation, solver, alpha, learning_rate_init, max_iter	Network architecture, activation, optimizer, regularization, learning rate, iterations.

* Exact available parameters can vary by scikit-learn version and by classifier vs regressor. Use `help(Model.fit)` or the official API documentation for the exact version installed.

4. What model.fit() takes

Estimator family	Required	Common optional fit arguments
Most supervised regressors/classifiers	X, y	sample_weight=None (when supported)
KNeighborsRegressor / Classifier	X, y	Usually no extra fit argument
MLPRegressor / Classifier	X, y	Usually no extra fit argument
KMeans (unsupervised)	X	sample_weight=None
PCA (unsupervised)	X	No y
IsolationForest	X	sample_weight=None (supported in current versions)

Key rule

```
# Supervised learning
model.fit(X, y)

# Example with optional sample weights
model.fit(X, y, sample_weight=weights)

# Unsupervised learning
model.fit(X)
```

5. Regression vs classification

Task	Target	Examples	Start with
Regression	A number	Price, fuel cost, salary, demand	LinearRegression / RandomForestRegressor
Classification	A category	Spam, fraud, disease class, pass/fail	LogisticRegression / RandomForestClassifier
Clustering	No target	Customer groups, similar products	KMeans
Dimensionality reduction	No target	Compress many features	PCA
Anomaly detection	Usually no target	Find unusual transactions	IsolationForest

6. Common unsupervised and anomaly-detection models

Model	Import	Use	fit()
KMeans	from sklearn.cluster import KMeans	Group observations into K clusters.	fit(X, y=None, sample_weight=None)
DBSCAN	from sklearn.cluster import DBSCAN	Density-based clustering; useful for irregular clusters/noise	fit(X, y=None)
AgglomerativeClustering	from sklearn.cluster import AgglomerativeClustering	Hierarchical clustering.	fit(X, y=None)
PCA	from sklearn.decomposition import PCA	Reduce dimensionality while retaining variance.	fit(X, y=None)
TruncatedSVD	from sklearn.decomposition import TruncatedSVD	Dimensionality reduction, especially sparse matrices.	fit(X, y=None)
IsolationForest	from sklearn.ensemble import IsolationForest	Detect unusual/outlier observations.	fit(X, y=None, sample_weight=None)
OneClassSVM	from sklearn.svm import OneClassSVM	One-class anomaly detection.	fit(X, y=None, sample_weight=None)
LocalOutlierFactor	from sklearn.neighbors import LocalOutlierFactor	Local-density based outlier detection.	fit(X, y=None)

7. Practical examples

A. Your fuel-cost problem

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

X = data[["distance"]]
y = data["cost"]

model.fit(X, y)

pred = model.predict(X)          # predict every distance already in X
new_pred = model.predict([[6000]]) # predict one new distance
```

Because X contains only distance, the prediction input must have exactly one feature. `[[6000]]` is 2-D because scikit-learn expects rows × features.

B. Random Forest regression

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_test)
```

C. Classification

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_test)
```

8. Choosing a model quickly

If your problem looks like...	Try first	Then compare with
One/few numeric features and roughly linear	LinearRegression	Ridge, RandomForestRegressor
Nonlinear tabular regression	RandomForestRegressor	GradientBoostingRegressor, HistGradientBoostingRegressor
Categorical target	LogisticRegression	RandomForestClassifier, GradientBoostingClassifier
Small dataset + nonlinear boundary	SVC / KNeighborsClassifier	RandomForestClassifier
Need groups without labels	KMeans	DBSCAN, AgglomerativeClustering
Need fewer dimensions	PCA	TruncatedSVD
Need anomaly detection	IsolationForest	OneClassSVM, LOF

9. Evaluation metrics you should know

Problem	Metric	Import	Meaning
Regression	MAE	from sklearn.metrics import mean_absolute_error	Average absolute prediction error.
Regression	MSE	from sklearn.metrics import mean_squared_error	Average squared error; penalizes large errors more.
Regression	RMSE	from sklearn.metrics import root_mean_squared_error	Square root of MSE; same units as target.
Regression	R ²	from sklearn.metrics import r2_score	How much target variance is explained relative to a constant baseline.
Classification	Accuracy	from sklearn.metrics import accuracy_score	Fraction of predictions that are correct.
Classification	Precision	from sklearn.metrics import precision_score	Of predicted positives, how many were correct.
Classification	Recall	from sklearn.metrics import recall_score	Of actual positives, how many were found.
Classification	F1	from sklearn.metrics import f1_score	Harmonic mean of precision and recall.

10. Essential imports to memorize

```
# Regression
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

# Classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Unsupervised
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA, TruncatedSVD

# Anomaly detection
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor
```

11. The rule that prevents most beginner mistakes

Do not choose a model because it has more parameters. Start with the simplest model that matches the problem, establish a baseline, evaluate it on held-out data, and then try more complex models. For your fuel-cost example, start with LinearRegression and compare it against RandomForestRegressor and GradientBoostingRegressor.

Also separate **features (X)** from the **target (y)**. If cost is what you are predicting, do not put cost into X.